

Time within transactions

Transactions are *time-discrete*: the time does not change during a transaction. You can easily see this by opening a transaction and querying the current time multiple times:

```

forumbd=> BEGIN;
BEGIN
forumbd=*> SELECT CURRENT_TIME;
      current_time
-----
14:51:50.730287+01
(1 row)

forumbd=*> SELECT pg_sleep_for( '5 seconds' );
      pg_sleep_for
-----

(1 row)

forumbd=*> SELECT CURRENT_TIME;
      current_time
-----
14:51:50.730287+01
(1 row)

forumbd=*> ROLLBACK;
ROLLBACK

```

If you really need a time-continuous source, while running a transaction, you can use `clock_timestamp()`:

```

forumbd=> BEGIN;
BEGIN
forumbd=*> SELECT CURRENT_TIME, clock_timestamp()::time;
      current_time      | clock_timestamp
-----+-----
14:53:17.479177+01 | 14:53:22.152435
(1 row)

```

```

forumdb=#> SELECT pg_sleep_for( '5 seconds' );
pg_sleep_for
-----

(1 row)

forumdb=#> SELECT CURRENT_TIME, clock_timestamp()::time;
current_time      | clock_timestamp
-----+-----
14:53:17.479177+01 | 14:53:33.022884

forumdb=#> ROLLBACK;
ROLLBACK

```

How can we identify one transaction from another? Every transaction gets an identifier, as explained in the following section.

More about transaction identifiers – the XID wraparound problem

As already explained, each transaction is associated with a numeric identifier called *xid* (where the *x* stands for transaction and *id* stands for identifier). Such a counter is used with a *modulo* 2^{31} operation, so that for any current *xid* value, there are 2^{31} transactions in the future, i.e., with a higher value. On the other hand, there are 2^{31} transactions in the past, i.e., with a lower value. Therefore, the *xid* counter is a cyclic value.

PostgreSQL does not allow two transactions to share the same *xid* in any case. However, being an automatically incremented counter, *xid* will sooner or later do a wraparound, which means it will start counting over. This is known as the *xid wraparound problem*, and PostgreSQL does a lot of work to prevent this from happening, as you will see later. But if the database is near the wraparound, PostgreSQL will start claiming it in the logs with messages like the following:

```

WARNING: database "forumdb" must be vacuumed within 177009986
transactions
HINT: To avoid a database shutdown, execute a database-wide VACUUM in
"forumdb".

```

If you carefully read the warning message, you will see that the system is telling the system administrator that it will shut down as soon as it detects the risk of an `xid` wraparound. The reason is that, in such circumstances, data could be lost, so in order to prevent this, the system will automatically shut down if the `xid` wraparound approaches.

There is, however, a way to avoid this automatic shutdown, by forcing a cleanup by means of the PostgreSQL tool named `VACUUM`. As you will see later in this chapter, one of the capabilities of `VACUUM` is to *freeze* old tuples so as to prevent the side effects of the `xid` wraparound, therefore allowing the continuity of the database service. But what are the effects of the `xid` wraparound?

In order to understand such problems, we have to remember that every transaction is assigned a unique `xid`, and that the next assignable `xid` is obtained by incrementing the last one assigned by a single unit.

This means that a transaction with a higher `xid` has started later than a transaction with a lower `xid`, even if the two transactions could be running in parallel. Since every tuple, in every table, stores the transaction identifier that created the tuple (in the `xmin` hidden field), a tuple with a higher `xid` must have been created after a tuple with a lower creation `xid`.

But when the `xid` overflows and, therefore, restarts its numbering from low numbers, transactions that started later will appear with a lower `xid` than already running transactions, and therefore, they will suddenly appear in the past. As a consequence, tuples with a higher transaction `xmin` would appear as if they have been created in the future, and therefore, there will be a mismatch of the temporal workflow and tuple storage.

To avoid the `xid` wraparound, PostgreSQL implements so-called *tuple freezing*: once a tuple is frozen, its `xmin` has to be considered always in the past with respect to any running transaction, even if its `xmin` is higher in value than any currently running transaction `xid`. Every tuple contains, in fact, a special bit of information that tells PostgreSQL if the tuple has been frozen or not.

Therefore, as the `xid` overflow approaches, `VACUUM` performs a wide freeze execution, marking all the tuples in the past as frozen, so that even if the `xid` restarts its counting from lower numbers, tuples already in the database will always appear in the past.



The xid counter starts at the special value of 3, keeping lower values for internal use only. Therefore, it is not possible to have a running transaction with an xid lower than 3. In older PostgreSQL versions, VACUUM literally removed the xmin value of the tuples to freeze substituting its value with the special value 2, which, being lower than the minimum usable value of 3, indicated that the tuple was in the past. However, when a forensic analysis is required, having the original xmin is valuable, and therefore, PostgreSQL now uses a status bit to indicate whether the tuple has been frozen.

Virtual and real transaction identifiers

Being such an important resource, PostgreSQL is smart enough to avoid wasting transaction identifier numbers. In particular, when a transaction is initiated, the cluster uses a *virtual xid*, something that works like an xid but is not obtained from the transaction identifier counter. This way, every transaction does not consume an xid number from the very beginning, thus reducing the need for a freeze. Once the transaction has done some work that involves data manipulation and changes, the virtual xid is transformed into a *real xid*, that is, one obtained from the xid counter. In other words, any “read-only” workload will not consume a transaction identifier, while any workload that involves writes on data will.

Thanks to this extra work, PostgreSQL does not waste transaction identifiers on those transactions that do not strictly require strong identification. For example, there is no need to waste an xid on a transaction block like the following:

```
forumdb=> BEGIN;  
BEGIN  
forumdb=> ROLLBACK;  
ROLLBACK
```

Since the preceding transaction does nothing at all, why should PostgreSQL involve all the xid machinery? There is no reason to use an xid that will not be attached to any tuple in the database and, therefore, not interfere with any active snapshot.

There is, however, an important thing to note: the usage of the `txid_current()` function always materializes an `xid` even if the transaction has not got one yet. For that reason, PostgreSQL provides another introspection function named `txid_current_if_assigned()`, which returns `NULL` if the transaction is still in the *virtual xid* space and, therefore, has not done any writable work yet. It is important to note that PostgreSQL will not assign a real `xid` unless the transaction has manipulated some data, and this can easily be proven with a workflow like the following one:

```

forumbd=> BEGIN;
BEGIN
forumbd=> SELECT txid_current_if_assigned();
txid_current_if_assigned
-----
(1 row)

forumbd=> SELECT count(*) FROM tags;
count
-----
      7
(1 row)

forumbd=> SELECT txid_current_if_assigned();
txid_current_if_assigned
-----
(1 row)

forumbd=> UPDATE tags SET tag = upper( tag );
UPDATE 7
forumbd=> SELECT txid_current_if_assigned();
txid_current_if_assigned
-----
4837

```

```
(1 row)

forumdb=> SELECT txid_current();
 txid_current
-----
          4837
(1 row)

forumdb=> ROLLBACK;
ROLLBACK
```

At the beginning of the transaction, there is no xid assigned, and in fact, `txid_current_if_assigned()` returns `NULL`. Even after a data read (that is, `SELECT`), the xid has not been assigned. However, as soon as the transaction performs some write activity (for example, an `UPDATE`), the xid is assigned, and the results of both `txid_current_if_assigned()` and `txid_current()` are the same.

Multi-version concurrency control

What happens if two transactions, either implicit or explicit, try to perform conflicting changes over the same data? PostgreSQL must ensure the data is always consistent, and therefore, it must have a way to *lock* (that is, block and protect) data subject to conflicting changes. Locks are a heavy mechanism that limits the concurrency of the system: the more locks you have, the more your transactions will wait to acquire the lock. To mitigate this problem, PostgreSQL implements MVCC, a well-known technique used in enterprise-level databases.

MVCC dictates that, instead of modifying an existing tuple within the database, the system has to replicate the tuple, apply the changes, and invalidate the original one. You can compare this to the copy-on-write mechanism used in operating filesystems such as ZFS.

To better understand what this means, let's assume the categories table has three tuples, and that we update one of them, to alter its description.

What happens is that a new tuple, derived from the one we are going to apply UPDATE to, is inserted into the table, and the original one is invalidated:



Figure 11.1: Updating a tuple creates a new tuple and invalidates the old one

Why are PostgreSQL and MVCC dealing with this extra work instead of doing an in-place update of the tuple? The reason is that this way, a database can cope with multiple versions of the same tuple, and every version is valid within a specific time window. This means that fewer locks are required to modify the data, since the database is able to handle multiple versions of the same data at the same time, and different transactions will see potentially different values.

For MVCC to work properly, PostgreSQL must handle the concept of *snapshots*: a snapshot indicates the time window in which a certain transaction is allowed to perceive data. A snapshot is, fundamentally, the range of transaction xids that define the boundaries of data available to a current transaction: every row in the database labeled with an xid within the range will be perceivable and usable by the current transaction. In other words, every transaction *sees* a dedicated subset of all the available data in the database. The MVCC machinery and the tests to decide if a tuple is visible or not are much more complex than the above description, but the idea at its core is as explained above.

The special function `txid_current_snapshot()` returns the minimum and maximum transaction identifiers that define the current transaction time boundaries. It becomes quite easy to demonstrate the concept with a couple of parallel sessions.

In the first session, let's run an explicit transaction, extract the identifier and the snapshot for future reference, and perform an operation:

```
-- session 1
```

```

forumbd=> BEGIN;
BEGIN
forumbd=> SELECT txid_current(), txid_current_snapshot();
  txid_current | txid_current_snapshot
-----+-----
      4928 | 4928:4928:
(1 row)

forumbd=> UPDATE tags SET tag = lower( tag );
UPDATE 5

```

As you can see in the preceding example, the transaction is number 4928, and its snapshot is bounded to itself, meaning that the transaction will see everything has been already consolidated in the database.

Now, let's pause this session for a moment and open another one connected to the same database – perform a single INSERT statement that is wrapped in an implicit transaction and get back the information about its xid:

```

forumbd=> INSERT INTO tags( tag ) VALUES( 'KDE' ) RETURNING txid_
current();
  txid_current
-----
      4929
(1 row)

```

The single-shot transaction has been assigned xid 4929, which is, of course, the very next xid available after the former explicit transaction (the system runs no other concurrent transactions to make it simpler to follow the numbering).

Go back to the first session and, again, inspect the information about the transaction snapshot:

```

-- session 1
forumbd=> SELECT txid_current(), txid_current_snapshot();
  txid_current | txid_current_snapshot
-----+-----
      4928      | 4928:4930:
(1 row)

```

This time, the transaction has grown its snapshot from itself to transaction 4930, which has not yet been started (`txid_current_snapshot()` reports its upper bound as non-inclusive). In other words, the current transaction now sees data consolidated even from a transaction that began after it, 4929. This can be even more explicit if the transaction queries the table:

```
-- session 1
forumbd=> SELECT xmin, tag FROM tags;
      xmin      | tag
-----+-----
4928 | linux
4928 | bsd
4928 | java
4928 | perl
4928 | raku
4929 | KDE
(6 rows)
```

As you can see, all the tuples but the last have been generated by the current transaction, and the last has been generated by xid 4929. But the preceding transaction is just a part of the story; while the first transaction is still incomplete, let's inspect the same table from another parallel session:

```
forumbd=> SELECT xmin, tag FROM tags;
      xmin      | tag
-----+-----
4922 | linux
4923 | BSD
4924 | Java
4925 | Perl
4926 | Raku
4929 | KDE
(6 rows)
```

All but the last tuple have different descriptions and, most notably, a different value for `xmin` from what transaction 4928 is seeing. What does it mean? It means that while the table has undergone an almost full rewrite of every tuple (an `UPDATE` on all but the last tuples), other concurrent transactions can still get access to the data in the table without having been blocked by a lock. This is the essence of MVCC: every transaction perceives a different view of the storage, and the view is valid depending on the time window (snapshot) associated with the transaction.

Sooner or later, the data on the storage has to be consolidated, and therefore, when transaction 4928 completes the COMMIT of its work, the data in the table will become the truth that every transaction from then on will perceive:

```
-- session 1
forumdb=> COMMIT;
COMMIT

-- out from the transaction now
-- we all see consolidated data
forumdb=> SELECT xmin, tag FROM tags;
      xmin      | tag
-----+-----
4928 | linux
4928 | bsd
4928 | java
4928 | perl
4928 | raku
4929 | KDE
(6 rows)
```

MVCC does not always prevent the usage of locks: if two or more concurrent transactions start manipulating the same set of data, the system has to apply ordered changes and, therefore, must force a lock on every concurrent transaction so that only one can proceed. It is quite simple to prove this with two parallel sessions similar to the preceding one:

```
-- session 1
forumdb=> BEGIN;
BEGIN
forumdb=> SELECT txid_current(), txid_current_snapshot();
      txid_current | txid_current_snapshot
-----+-----
4930 | 4930:4930:
(1 row)

forumdb=> UPDATE tags SET tag = upper( tag );
UPDATE 6
```